

Rotación LL en Árboles AVL

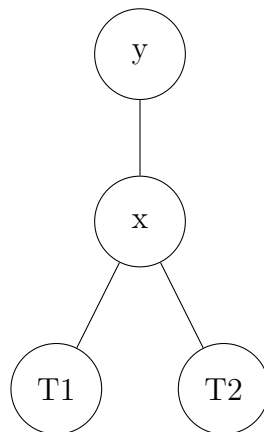
Rotación LL: Explicación Paso a Paso

¿Cuándo se aplica?

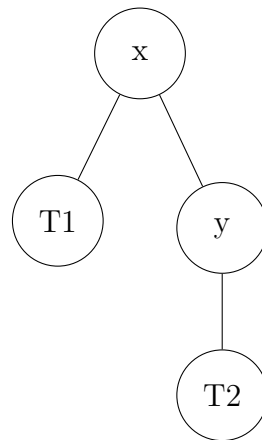
La rotación LL (simple a la derecha) se aplica cuando:

- El subárbol izquierdo del nodo desbalanceado tiene un hijo izquierdo.
- El balance del nodo es $+2$.
- Es una rotación para reducir altura del lado izquierdo.

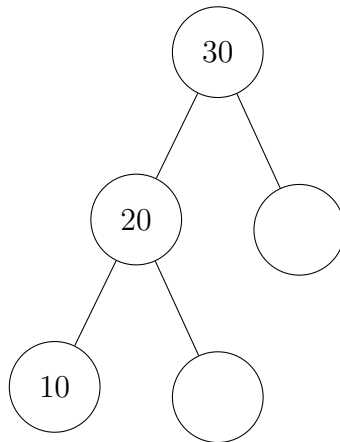
Estructura general antes de la rotación



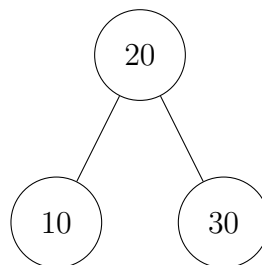
Estructura después de la rotación



Ejemplo con valores reales: Antes de la rotación



Ejemplo con valores reales: Después de la rotación



Código Java de la rotación LL

```
1 // Realiza rotacion simple a la derecha sobre el nodo 'y'
2 public Nodo rotarDerecha(Nodo y) {
3     Nodo x = y.izquierdo;      // x se vuelve la nueva raiz del
        subarbol
4     Nodo T2 = x.derecho;        // T2 es el subarbol entre x e y
5
6     // Realiza la rotacion
7     x.derecho = y;              // x se convierte en padre de y
8     y.izquierdo = T2;          // T2 queda como hijo izquierdo de y
9
10    return x;                   // x es la nueva raiz del subarbol
        balanceado
11 }
```

Explicación de cada línea del código

- `Nodo x = y.izquierdo;`
Se identifica el hijo izquierdo del nodo desbalanceado.
- `Nodo T2 = x.derecho;`
Se guarda el subárbol que quedaría .entre” los dos nodos tras la rotación.
- `x.derecho = y;`
Se realiza la rotación: x sube, y baja a la derecha.
- `y.izquierdo = T2;`
Se conecta T2 como nuevo hijo izquierdo de y (puede ser null o un subárbol real).
- `return x;`
Se retorna la nueva raíz del subárbol.

Ejemplo en Java con main

```
1 // Clase Nodo
2 class Nodo {
3     int valor;
4     Nodo izquierdo, derecho;
5
6     Nodo(int val) {
7         valor = val;
8         izquierdo = derecho = null;
9     }
10 }
11 }
```

```

12 // Clase principal
13 public class ArbolAVL {
14
15     // Rotacion simple a la derecha
16     public static Nodo rotarDerecha(Nodo y) {
17         Nodo x = y.izquierdo;
18         Nodo T2 = x.derecho;
19
20         x.derecho = y;
21         y.izquierdo = T2;
22
23         return x;
24     }
25
26     public static void main(String[] args) {
27         // Paso 1: Insertamos nodos de forma que se genere un
28             // desbalance tipo LL
29         Nodo raiz = new Nodo(30);
30         raiz.izquierdo = new Nodo(20);
31         raiz.izquierdo.izquierdo = new Nodo(10);
32
33         System.out.println("Antes de rotar:");
34         imprimirEnOrden(raiz); // Deber a imprimir: 10 20 30
35
36         // Paso 2: Aplicamos la rotacion LL (derecha)
37         raiz = rotarDerecha(raiz);
38
39         System.out.println("\nDespu s de rotar:");
40         imprimirEnOrden(raiz); // Deber a seguir imprimiendo: 10 20
41             // 30 pero balanceado
42     }
43
44     // Metodo auxiliar para recorrer el arbol en orden
45     public static void imprimirEnOrden(Nodo nodo) {
46         if (nodo != null) {
47             imprimirEnOrden(nodo.izquierdo);
48             System.out.print(nodo.valor + " ");
49             imprimirEnOrden(nodo.derecho);
50         }
51     }
52 }

```